



ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Name: ETHO MBA Steeve Rayanne

ID: 25979

ASSIGNMENT :

1/ The functional requirements :

ID	Functional Requirement
FR1	Employee Management: The system shall allow adding, updating, viewing, and deleting employee details (e.g., ID, name, role, base salary).
FR2	Payment Processing: The system shall calculate and process salary payments, taking into account base pay, bonuses, and tax/deductions.
FR3	Reporting: The system shall generate summary reports detailing total salary expenses and employee lists.

2/ Java Implementation:

I used Eclipse.

```
package com.auca;  
  
import java.util.ArrayList;  
import java.util.List;
```

```

public class EmployeeManagementSystem {

    public static class Employee {
        private String id;
        private String name;
        private double baseSalary;

        public Employee(String id, String name, double baseSalary) {
            this.id = id;
            this.name = name;
            this.baseSalary = baseSalary;
        }

        public String getId() { return id; }
        public String getName() { return name; }
        public double getBaseSalary() { return baseSalary; }
    }

    private List<Employee> employees = new ArrayList<>();

    // FR-01: Employee Management
    public boolean addEmployee(String id, String name, double baseSalary) {
        if (id == null || id.isEmpty() || baseSalary < 0) {
            return false;
        }
        employees.add(new Employee(id, name, baseSalary));
        return true;
    }

    // FR-02: Payment Processing
    public double processPayment(String employeeId, double bonus) {
        if (bonus < 0) {
            return -1.0;
        }

        Employee emp = null;
        for (Employee e : employees) {
            if (e.getId().equals(employeeId)) {
                emp = e;
                break;
            }
        }

        if (emp == null) {
            return -1.0;
        }

        double grossSalary = emp.getBaseSalary() + bonus;
        double taxRate;

        // Fault injected intentionally (>= instead of >)
        if (grossSalary >= 5000) {
            taxRate = 0.20;
        } else {

```

```

        taxRate = 0.10;
    }

    return grossSalary * (1 - taxRate);
}

// FR-03: Reporting
public String generateSummaryReport() {
    double totalExpense = 0;
    for (Employee e : employees) {
        totalExpense += e.getBaseSalary();
    }
    return "Total Employees: " + employees.size() + ", Total Base Expense: $" +
totalExpense;
}

// ENTRY POINT NEEDED BY ECLIPSE
public static void main(String[] args) {
    EmployeeManagementSystem ems = new EmployeeManagementSystem();

    // Add test employee
    ems.addEmployee("EMP001", "Steeve Rayanne", 4000);

    // Execute test case
    double actualNetPay = ems.processPayment("EMP001", 1000);
    double expectedNetPay = 4500.0;

    System.out.println("=== TEST EXECUTION RESULTS ===");
    System.out.println("Expected Net Pay: $" + expectedNetPay);
    System.out.println("Actual Net Pay:    $" + actualNetPay);

    if (actualNetPay == expectedNetPay) {
        System.out.println("TEST RESULT: PASS");
    } else {
        System.out.println("TEST RESULT: FAIL");
    }

    System.out.println("\n=== SUMMARY REPORT ===");
    System.out.println(ems.generateSummaryReport());
}
}

```

3/ General Test Information

Parameter	Information
Institution	Adventist University of Central Africa (AUCA)
Student Name	ETHO MBA Steeve Rayanne
Student ID	25979
Test Date	September 22, 2026
System Date	September 22, 2026
Hardware	Standard PC / x86 Workstation
Software	Java SE Development Kit (JDK 17 or higher), Eclipse IDE, JUnit 5, JaCoCo Coverage Tool

4/ Black-Box Test Case

Black-box testing focuses on testing inputs and expected outputs without looking at the internal source code structure.

Field	Details
Test Case Number	TC-BB-01
Test Stage	Functionality
Tester	Steeve Rayanne
Incident Number	INC-2026-001

Field	Details
Requirement(s) Tested	FR2 (Payment Processing) — Verify system processes payments correctly based on base salary and bonus rates.
Test Description	Verify payment calculation for employee with a gross salary of \$5,000 (Boundary Value Analysis for payment processing).
Input Specifications	Employee ID: "EMP001" (Base Salary: \$4,000), Bonus: \$1,000, Gross Pay: \$5,000
Procedural Steps	1. Instantiate EmployeeManagementSystem. 2. Call addEmployee("EMP001", "Steeve Rayanne", 4000). 3. Call processPayment("EMP001", 1000). 4. Compare return value with expected net pay.
Expected Result	Expected Net Pay = $\$5,000 \times (1 - 0.10) = \$4,500.00$. Pass criteria: Returned value equals \$4,500.00.
Actual Result	Returned value from processPayment("EMP001", 1000) was \$4,000.00.
Discrepancy	Expected \$4,500.00, but received \$4,000.00 due to wrong tax tier application (20% applied instead of 10%).
Result	FAIL

4/ White Box Test Case

White-box testing examines the internal structure, branches, and code logic.

Field	Details
Test Case Number	TC-WB-01
Test Stage	Unit
Tester	Steeve Rayanne
Incident Number	INC-2026-002
Requirement(s) Tested	Processing Payments: Structural coverage for conditional evaluation if (grossSalary >= 5000).
Test Description	White-box branch coverage and fault-injection path testing for conditional tax selection in processPayment.
Input Specifications	Employee ID: "EMP002", Base Salary: \$5,000.00, Bonus: \$0.00, Gross Pay: \$5,000.00
Procedural Steps	1. Execute addEmployee("EMP002", "Daniel Lawson", 5000). 2. Invoke processPayment("EMP002", 0). 3. Inspect condition evaluation path: Verify whether condition evaluates to false (10% tax rate) as specified by requirements.
Expected Result	Boundary value 5000.0 should evaluate to false on condition grossSalary > 5000, selecting taxRate = 0.10 and yielding output \$4,500.00.
Actual Result	The condition if (grossSalary >= 5000) evaluated to true, forcing execution of taxRate = 0.20 branch.
Discrepancy	Fault injected in relational operator (>= instead of >). Cause of failure identified: relational operator fault in decision branch.

Field	Details
Result	FAIL